Lesson 1: **Python variables and data types**

```
gate_name = "AND"
number_of_inputs = 2
delay = 1.5
is_enabled = True

print(1)
print(type(gate_name))
print(type(number_of_inputs))
print(type(delay))
print(type(is_enabled))
print()
print("Gate:", gate_name)
print("Number of Inputs:", number_of_inputs)
print("Delay:", delay)
print("Is Enabled:", is_enabled)
print()
gate_name = "XOR"
number_of_inputs = 2
delay = 3.2

print(2)
print("Gate:", gate_name)
print("Number of Inputs:", number_of_inputs * 2)
print("Delay:", delay + 1)
print()

gate_name = "NAND1"
stuck_at_0 = gate_name + "_SA0"
stuck_at_1 = gate_name + "_SA1"
faults = [stuck_at_0, stuck_at_1]

print(3)
print(type(faults))
print()
print("Gate:", gate_name)
print("Fault hypotheses:", faults)
print("Total hypotheses:", len(faults))
print()
```

```
PS C:\Users\Lenovo\Learn Python> py lesson1.py
1
<class 'str'>
<class 'int'>
<class 'float'>
<class 'bool'>

Gate: AND
Number of Inputs: 2
Delay: 1.5
Is Enabled: True

2
Gate: XOR
Number of Inputs: 4
Delay: 4.2

3
<class 'list'>

Gate: NAND1
Fault hypotheses: ['NAND1_SA0', 'NAND1_SA1']
Total hypotheses: 2
```

inspect a variable's type using type()

Variable	Type
gate_name	str
number_of_inputs	int
delay	float
is_enabled	bool
faults	list

Lesson 2: **Python operators and expressions**

```
x = 10
print(1)
print("x > 5", x > 5)    # True
print("x == 10", x == 10)  # True
print("x != 7", x != 7)   # True
print("x < 5", x < 5)    # False
print()

voltage = 1
enabled = True
print(2)
print("voltage=", voltage)
print("enabled=", enabled)
print("voltage == 1 and enabled", voltage == 1 and enabled)  # True
print("voltage == 0 or enabled", voltage == 0 or enabled)   # True
print("not enabled", not enabled)                # False
print()

column = 10
number_of_banks = 4

bank = column % number_of_banks
local_column = column // number_of_banks
print(3)
print("Bank:", bank)
print("Local Column:", local_column)
print()

# Arithmetic operators
a = 17
b = 5
print(4)
print("Addition:", a + b)
print("Division:", a / b)
print("Floor division:", a // b)
print("Remainder:", a % b)
print()
# Comparisons
print("a greater than b:", a > b)
print("a equals b:", a == b)
print()

# Logical operators
ready = True
has_error = False
print(5)
print("Can proceed:", ready and not has_error)
print()

def calculate_address(row, column):
    bank = column % 4
    local_address = 64 * row + column // 4
    physical_address = 16384 * bank + local_address

    return bank, local_address, physical_address

row = 2
column = 10

bank, local, physical = calculate_address(row, column)
print(6)
print("Bank:", bank)
print("Local address:", local)
print("Physical address:", physical)
print()
```

```
PS C:\Users\Lenovo\Learn Python> py lesson2.py
1
x > 5 True
x == 10 True
x != 7 True
x < 5 False

2
voltage= 1
enabled= True
voltage == 1 and enabled True
voltage == 0 or enabled True
not enabled False

3
Bank: 2
Local Column: 2

4
Addition: 22
Division: 3.4
Floor division: 3
Remainder: 2

a greater than b: True
a equals b: False

5
Can proceed: True

6
Bank: 2
Local address: 130
Physical address: 32898
```

Lesson 3: Conditional statements

```
#example1
temperature = 105
if temperature > 100:
    print(1)
    print("High temperature")
    print()

#example2
probability = 0.35

print(2)
if probability >= 0.5:
    prediction = 1
    print("Class 1 selected")
else:
    prediction = 0
    print("Class 0 selected")

print("Prediction:", prediction)
print()

#example3
score = 85
if score >= 90:
    grade = "A"
elif score >= 75:
    grade = "B"
else:
    grade = "C"
print(3)
print(grade)
print()

#example4
def classify_binary(probability):
    if probability >= 0.70:
        return 1
    elif probability < 0.30:
        return 0
    else:
        return "Uncertain"

print(4)
for p in [0.15, 0.50, 0.85]:
    print(p, "->", classify_binary(p))
print()

#example5
print(5)
def evaluate_fault(fault_name, detected):
    if detected:
        status = "DETECTED"
    else:
        status = "UNDETECTED"

    print(f"{fault_name}: {status}")

evaluate_fault("N1_SA0", True)
evaluate_fault("N2_SA1", False)
print()
```

```
PS C:\Users\Lenovo\Learn Python> py lesson3.py
1
High temperature

2
Class 0 selected
Prediction: 0

3
B

4
0.15 -> 0
0.5 -> Uncertain
0.85 -> 1

5
N1_SA0: DETECTED
N2_SA1: UNDETECTED
```

Lesson 4: Loops

```
#example1
gates = ["AND", "OR", "NOT"]
print(1)
print("-----")
for gate in gates:
    print(gate)
print()

#example2
print(2)
print("-----")
for i in range(4):
    print(i)
print()

#example3
count = 0
print(3)
print("-----")
while count < 3:
    print(count)
    count += 1
print()

#example4
gates = ["N1", "N2"]
print(4)
print("-----")
for gate in gates:
    print(gate + "_SA0")
    print(gate + "_SA1")
print()

#example5
gates = ["AND1", "OR1", "NOT1"]
print(5)
print("-----")
for gate in gates:
    print("Processing:", gate)
    print(gate + "_SA0")
    print(gate + "_SA1")

print("Fault generation complete.")
print()

#example6
print(6)
print("-----")
count = 0

while count < 4:
    print("Iteration:", count)
    count += 1

print("Loop finished.")
print()

#example7
probabilities = [0.15, 0.62, 0.91, 0.38, 0.77]
predictions = []
print(7)
print("-----")
for probability in probabilities:
    if probability >= 0.5:
        prediction = 1
    else:
        prediction = 0

    predictions.append(prediction)

print("Predictions:", predictions)
print()

#example8
```

```
PS C:\Users\Lenovo\Learn Python> py lesson4.py
1
-----
AND
OR
NOT

2
-----
0
1
2
3

3
-----
0
1
2

4
-----
N1_SA0
N1_SA1
N2_SA0
N2_SA1
```

```
5
-----
Processing: AND1
AND1_SA0
AND1_SA1
Processing: OR1
OR1_SA0
OR1_SA1
Processing: NOT1
NOT1_SA0
NOT1_SA1
Fault generation complete.

6
-----
Iteration: 0
Iteration: 1
Iteration: 2
Iteration: 3
Loop finished.

7
-----
Predictions: [0, 1, 1, 0, 1]

8
-----
Total fault hypotheses: 8
N1_SA0
N1_SA1
N2_SA0
N2_SA1
N3_SA0
N3_SA1
OUT_SA0
OUT_SA1
```

```
nets = ["N1", "N2", "N3", "OUT"]
faults = []
print(8)
print("-----")
for net in nets:
    faults.append(net + "_SA0")
    faults.append(net + "_SA1")

print("Total fault hypotheses:", len(faults))

for fault in faults:
    print(fault)
print()

#example9
print(9)
print("-----")
for i in range(1, 4):
    print(i)
print()
```

```
9
-----
1
2
3
```

Lesson 5: Python functions

```python
#example1
def average(a, b):
    result = (a + b) / 2
    return result

print(1)
print("-----")
value = average(10, 20)
print("mean of 10 and 20 is:", value)
print()

#example2
def square(x):
    return x * x
print(2)
print("-----")
answer = square(4)
print("square of 4 is:", answer)
print()

#example3

def calculate_power(voltage, current):
    power = voltage * current
    return power

v = 5
i = 2

p = calculate_power(v, i)

print(3)
print("-----")
print("Voltage:", v, "V")
print("Current:", i, "A")
print("Power:", p, "W")
print()

#example4

import math

def linear_score(x, weight, bias):
    return x * weight + bias

def sigmoid(z):
    return 1 / (1 + math.exp(-z))

def predict(x, weight, bias):
    score = linear_score(x, weight, bias)
    probability = sigmoid(score)

    if probability >= 0.5:
        prediction = 1
    else:
        prediction = 0

    return probability, prediction

probability, prediction = predict(2, 1.0, -1.0)
print(4)
print("-----")
print("Probability:", round(probability, 4))
print("Prediction:", prediction)
print()

#example5

def generate_faults_for_net(net):
    return [net + "_SA0", net + "_SA1"]

def generate_faults_for_circuit(nets):
    all_faults = []
```

```
PS C:\Users\Lenovo\Learn Python> py lesson5.py
1
-----
mean of 10 and 20 is: 15.0

2
-----
square of 4 is: 16

3
-----
Voltage: 5 V
Current: 2 A
Power: 10 W

4
-----
Probability: 0.7311
Prediction: 1

5
-----
Fault count: 6
Fault list: ['N1_SA0', 'N1_SA1', 'N2_SA0', 'N2_SA1', 'OUT_SA0', 'OUT_SA1']
```

```
    for net in nets:
        net_faults = generate_faults_for_net(net)
        all_faults.extend(net_faults)

    return all_faults

nets = ["N1", "N2", "OUT"]
faults = generate_faults_for_circuit(nets)

print(5)
print("-----")
print("Fault count:", len(faults))
print("Fault list:", faults)
print()
```

Lesson 6: Python collection

```
#example1
# List: ordered fault hypotheses
faults = ["N1_SA0", "N1_SA1", "N1_SA0"]

# Set: remove duplicate entries
unique_faults = set(faults)

# Tuple: fixed row-column coordinate
coordinate = (2, 10)

# Dictionary: signal names and logic values
signals = {"A": 1, "B": 0}

signals["B"] = 1
print("example1")
print("Original faults:", faults)
print()
print("Unique faults:", unique_faults)
print("Unique fault count:", len(unique_faults))
print()
print("Coordinate:", coordinate)
print()
print("Updated signals:", signals)
print()

#example2

# Each tuple contains (input feature, class label)
dataset = [
    (0.2, 0),
    (0.7, 1),
    (0.9, 1),
    (0.3, 0)
]

predictions = [0, 1, 1, 1]

correct = 0

for i in range(len(dataset)):
    actual_label = dataset[i][1]
    predicted_label = predictions[i]

    if actual_label == predicted_label:
        correct += 1

accuracy = correct / len(dataset)

metrics = {"accuracy": accuracy}
print("example2")
print("Correct predictions:", correct)
print("Metrics:", metrics)
print()

#example3

signals = {
    "A": 1,
    "B": 0
}

faults = [
    {"net": "A", "stuck_at": 0},
    {"net": "B", "stuck_at": 1}
]

print("example3")

for fault in faults:
    net = fault["net"]
    stuck_value = fault["stuck_at"]

    observed_value = signals[net]
```

```
PS C:\Users\Lenovo\Learn Python> py lesson6.py
example1
Original faults: ['N1_SA0', 'N1_SA1', 'N1_SA0']

Unique faults: {'N1_SA1', 'N1_SA0'}
Unique fault count: 2

Coordinate: (2, 10)

Updated signals: {'A': 1, 'B': 1}

example2
Correct predictions: 3
Metrics: {'accuracy': 0.75}

example3
A detected: True
B detected: True
```

```
    if observed_value != stuck_value:
        detected = True
    else:
        detected = False

    print(net, "detected:", detected)
```

Lesson 7: **Indexing, slicing, and nested collections**

```
#example1

# A list of circuit net names
nets = ["A", "B", "Y", "Z", "OUT"]
print("example1")
print("First net:", nets[0])
print("Last net:", nets[-1])
print("Middle section:", nets[1:4])
print()


#example2

# A tiny ML dataset: (feature, label)
dataset = [
    (0.2, 0),
    (0.7, 1),
    (0.9, 1)
]

sample = dataset[1]
print("example2")
print("Selected sample:", sample)
print("Feature:", sample[0])
print("Label:", sample[1])
print()

#example3

# Each row contains [feature_1, feature_2, label]
dataset = [
    [1.2, 3.4, 0],
    [2.1, 4.5, 1],
    [0.8, 2.7, 0],
    [3.0, 5.1, 1]
]

# Select the second sample
sample = dataset[1]

# Separate its features and label
features = sample[:2]
label = sample[2]
print("example3")
print("Selected sample:", sample)
print("Features:", features)
print("Label:", label)

# Select the first two samples
batch = dataset[:2]
print("Batch size:", len(batch))
print()

#example4

gates = [
    ["AND", "A", "B", "N1"],
    ["OR", "N1", "C", "N2"],
    ["NOT", "N2", None, "OUT"]
]

print("example4")
for gate in gates:
    gate_type = gate[0]
    output_net = gate[3]

    print("Gate:", gate_type)
    print("Output:", output_net)

    if gate_type == "NOT":
        print("This gate has one input.")
print()
```

```
PS C:\Users\Lenovo\Learn Python> py lesson7.py
example1
First net: A
Last net: OUT
Middle section: ['B', 'Y', 'Z']

example2
Selected sample: (0.7, 1)
Feature: 0.7
Label: 1

example3
Selected sample: [2.1, 4.5, 1]
Features: [2.1, 4.5]
Label: 1
Batch size: 2

example4
Gate: AND
Output: N1
Gate: OR
Output: N2
Gate: NOT
Output: OUT
This gate has one input.

example5
['AND1', 'A', 'B', 'Y']
AND1
Y
Z
Z
Traceback (most recent call last):
  File "C:\Users\Lenovo\Learn Python\lesson7.py", line 86, in <module>
    print(circuit[1][3])
          ~~~~~~~~~~^^^
IndexError: list index out of range
```

```
#example5
circuit = [
    ["AND1", "A", "B", "Y"],
    ["NOT1", "Y", "Z"]
]
print("example5")
print(circuit[0])
print(circuit[0][0])
print(circuit[0][3])
print(circuit[1][2])
print(circuit[-1][-1])
print(circuit[1][3])
print()
```

Lesson 8: Classes and objects (OOP)

```python
#example1

class LogicGate:
    print()
    print("example1")
    def __init__(self, name, gate_type):
        self.name = name
        self.gate_type = gate_type

    def describe(self):
        print(f"{self.name} is an {self.gate_type} gate.")

gate_a = LogicGate("G1", "AND")
gate_b = LogicGate("G2", "OR")

gate_a.describe()
gate_b.describe()
print()

#example2

class LogicGate:
    def __init__(self, name, gate_type):
        self.name = name
        self.gate_type = gate_type

    def evaluate(self, a, b=None):
        if self.gate_type == "AND":
            return a & b
        elif self.gate_type == "OR":
            return a | b
        elif self.gate_type == "NOT":
            return 1 - a
        else:
            raise ValueError("Unsupported gate type")

and_gate = LogicGate("G1", "AND")
or_gate = LogicGate("G2", "OR")
not_gate = LogicGate("G3", "NOT")

print("example2")
print("and of 1 and 0 is", and_gate.evaluate(1, 0))
print("or of 1 and 0 is", or_gate.evaluate(1, 0))
print("not of 1 is", not_gate.evaluate(1))
print()

#example3

import math

class LogisticModel:
    def __init__(self, weight, bias):
        self.weight = weight
        self.bias = bias

    def predict_probability(self, x):
        z = x * self.weight + self.bias
        return 1 / (1 + math.exp(-z))

    def predict(self, x):
        probability = self.predict_probability(x)
        return 1 if probability >= 0.5 else 0

model = LogisticModel(weight=1.0, bias=-1.0)

x = 2.0

print("example3")
print("Probability:", round(model.predict_probability(x), 4))
print("Prediction:", model.predict(x))
print()
```

```
PS C:\Users\Lenovo\Learn Python> py lesson8.py

example1
G1 is an AND gate.
G2 is an OR gate.

example2
and of 1 and 0 is 0
or of 1 and 0 is 1
not of 1 is 0

example3
Probability: 0.7311
Prediction: 1
```

Lesson 9: **Modules, imports, and project architecture**

Example 1:

faults.py

```
def generate_faults(net):
    return [net + "_SA0", net + "_SA1"]


def count_faults(fault_list):
    return len(fault_list)
```

```
PS C:\Users\Lenovo\Learn Python\lesson9.py> py main.py
Faults: ['N1_SA0', 'N1_SA1', 'N2_SA0', 'N2_SA1', 'OUT_SA0', 'OUT_SA1']
Count: 6
```

main.py

```
from faults import generate_faults, count_faults

nets = ["N1", "N2", "OUT"]
all_faults = []

for net in nets:
    all_faults.extend(generate_faults(net))

print("Faults:", all_faults)
print("Count:", count_faults(all_faults))
```

Example 2:

model.py

```
import math


def predict_probability(x, weight, bias):
    z = x * weight + bias
    return 1 / (1 + math.exp(-z))


def predict_class(probability, threshold=0.5):
    return 1 if probability >= threshold else 0
```

```
PS C:\Users\Lenovo\Learn Python\lesson9.py> py main1.py
Probability: 0.7311
Prediction: 1
```

main1.py

```
from model import predict_probability, predict_class

weight = 1.0
bias = -1.0
x = 2.0

probability = predict_probability(x, weight, bias)
prediction = predict_class(probability)

print("Probability:", round(probability, 4))
print("Prediction:", prediction)
```

Lesson 10: **Exceptions, debugging, and error handling**

```python
#example1
print("example1")
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero.")

print()

#example2
print("example2")
def calculate_average(values):
    if not values:
        raise ValueError("The values list cannot be empty.")

    return sum(values) / len(values)

def main():
    readings = [10, 20, 30]

    try:
        average = calculate_average(readings)
    except ValueError as error:
        print("Input error:", error)
    else:
        print("Average:", average)
    finally:
        print("Calculation attempt finished.")

if __name__ == "__main__":
    main()

print()

#example3
print("example3")
def validate_net(record):
    if "name" not in record:
        raise ValueError("Net record is missing 'name'.")

    if "value" not in record:
        raise ValueError("Net record is missing 'value'.")

    if record["value"] not in (0, 1):
        raise ValueError(
            f"Invalid binary value for net {record['name']}."
        )

    return True

net = {"name": "N1", "value": 1}

try:
    validate_net(net)
except ValueError as error:
    print("Netlist error:", error)
else:
    print("Net record is valid.")
print()

#example4

import numpy as np
print("example4")
weights = np.array([0.2, 0.5, 0.1])
features = np.array([1.0, 2.0])

print("Weights shape:", weights.shape)
print("Features shape:", features.shape)

if weights.shape != features.shape:
    raise ValueError(
        "Feature count must match weight count."
    )
```

```
PS C:\Users\Lenovo\Learn Python> py lesson10.py
example1
Cannot divide by zero.

example2
Average: 20.0
Calculation attempt finished.

example3
Net record is valid.

example4
Weights shape: (3,)
Features shape: (2,)
Traceback (most recent call last):
  File "C:\Users\Lenovo\Learn Python\lesson10.py", line 77, in <module>
    raise ValueError(
        "Feature count must match weight count."
    )
ValueError: Feature count must match weight count.
```

```
score = np.dot(weights, features)
print(score)

print()
```

Lesson 11: **Files, reading and writing data**

Example 1:

readings.txt

```
10
20
30
40
```

lesson11.py

```
def load_numbers(filename):
    numbers = []

    with open(filename, "r", encoding="utf-8") as file:
        for line in file:
            value = line.strip()

            if value:
                numbers.append(float(value))

    return numbers


def main():
    numbers = load_numbers("readings.txt")

    print("Loaded values:", numbers)
    print("Number of readings:", len(numbers))
    print("Average:", sum(numbers) / len(numbers))


if __name__ == "__main__":
    main()
```

```
PS C:\Users\Lenovo\Learn Python\lesson11> py lesson11.py
Loaded values: [10.0, 20.0, 30.0, 40.0]
Number of readings: 4
Average: 25.0
```

Example 2:

dataset.csv

```
feature1,feature2,label
1.0,2.0,0
1.5,1.0,0
7.0,8.0,1
8.0,7.5,1
```

```
PS C:\Users\Lenovo\Learn Python\lesson11> py load_dataset.py
Features:
 [[1.  2. ]
 [1.5 1. ]
 [7.  8. ]
 [8.  7.5]]
Labels: [0 0 1 1]
Feature matrix shape: (4, 2)
Label vector shape: (4,)
```

load_dataset.py

```
import csv
import numpy as np

def load_dataset(filename):
    features = []
    labels = []

    with open(filename, "r", newline="", encoding="utf-8") as file:
        reader = csv.DictReader(file)

        expected = {"feature1", "feature2", "label"}
        if not reader.fieldnames or not expected.issubset(reader.fieldnames):
            raise ValueError("CSV is missing required columns.")

        for row_number, row in enumerate(reader, start=2):
            try:
                x1 = float(row["feature1"])
                x2 = float(row["feature2"])
                label = int(row["label"])
            except (ValueError, TypeError) as error:
                raise ValueError(
                    f"Invalid data on CSV row {row_number}."
                ) from error

            if label not in (0, 1):
```

```
                raise ValueError(
                    f"Label on row {row_number} must be 0 or 1."
                )

            features.append([x1, x2])
            labels.append(label)

    X = np.array(features, dtype=float).reshape(-1, 2)
    y = np.array(labels, dtype=int)

    if len(y) == 0:
        raise ValueError("Dataset contains no samples.")

    return X, y

if __name__ == "__main__":
    X, y = load_dataset("dataset.csv")

    print("Features:\n", X)
    print("Labels:", y)
    print("Feature matrix shape:", X.shape)
    print("Label vector shape:", y.shape)
```

Example 3:

faults.txt

```
N1 SA0
N1 SA1
N2 SA0
N2 SA1
```

```
PS C:\Users\Lenovo\Learn Python\lesson11> py load_faults.py
{'net': 'N1', 'fault_type': 'SA0'}
{'net': 'N1', 'fault_type': 'SA1'}
{'net': 'N2', 'fault_type': 'SA0'}
{'net': 'N2', 'fault_type': 'SA1'}
Total faults: 4
```

load_faults.txt

```
def load_faults(filename):
    faults = []

    with open(filename, "r", encoding="utf-8") as file:
        for line_number, line in enumerate(file, start=1):
            parts = line.split()

            if not parts:
                continue

            if len(parts) != 2:
                raise ValueError(
                    f"Malformed fault on line {line_number}: {line.strip()!r}"
                )

            net, fault_type = parts

            if fault_type not in ("SA0", "SA1"):
                raise ValueError(
                    f"Unsupported fault type on line {line_number}: "
                    f"{fault_type}"
                )

            faults.append({
                "net": net,
                "fault_type": fault_type
            })

    return faults


if __name__ == "__main__":
    faults = load_faults("faults.txt")

    for fault in faults:
        print(fault)

    print("Total faults:", len(faults))
```